



Best Practices in Semestral Projects

Android Lecture

SW development best practices

- Use git and gitflow
- In a group of people: do code reviews
- Ensure your code is readable, uniformly formatted and free of unused classes/methods/variables - utilise static analysis tools (lint, detekt)
- Comments/Javadocs for complex classes/methods/code.
- Additionally (out of scope for semestral projects):
 - write tests (unit, UI),
 - write documentation - flows, complex algorithms, javadocs in code, user manuals, database structure, network communication diagrams, etc.,
 - create libraries for parts used in multiple projects (simplify maintenance, speed-up builds)

Android best practices

SDK Configuration

- **minSdk**: Defines the minimum supported Android version. When in doubt, use the recommended version. Avoid going too low unless necessary, as it increases development complexity.
- **targetSdk**: Set this to the latest supported version to align with the latest platform features and security updates.
- **compileSdk**: Always use the latest available version for compilation.

Android best practices

Technologies:

- UI: [Jetpack Compose](#) (preferred), optionally old way using XML layouts
- Dependency Injection: Handle dependencies manually or use [Koin](#) for simplicity
- Image loading: [Glide](#) or [Coil](#)
- Network communication: [Retrofit](#) or [Ktor](#)

Android best practices

Technologies for persistence:

- Simple primitive values: DataStore
- More complex data and lists: Room Database
- Backup and sharing across different devices/platforms: Server/Cloud services
- For specific usecases Files can be used (but database or datastore is preferred for most cases)

Android best practices

Resources:

- *Strings*: Store all application texts (e.g., button labels, messages) in the `strings.xml` file. This practice aids in maintenance and facilitates providing translations if needed.
- *Images*: Prefer using vector images in XML format for scalability. If using bitmap images, provide multiple sizes (e.g., hdpi, mdpi, xhdpi, etc.) to support various screen densities.
- Other resources: colours, dimensions, etc. should be defined as resources.

Android best practices

Navigation

- Test navigation thoroughly to ensure all back buttons and navigation flows work as expected.
- Options for implementing navigation:
 - **Recommended:** Navigation Component, suitable for both Compose and Fragments.
 - For navigation between activities: use `startActivity(intent)`.
 - For navigation between fragments: Navigation component, use of `FragmentManager` is possible, but discouraged

Android best practices

MVVM or MVI Architecture:

- Follow the **Model-View-ViewModel (MVVM)** or **MVI (Model-View-Intent)** pattern.
- Ensure that Activities, Fragments, and Composables contain no business logic. All business logic should reside in ViewModels and other components (e.g. communication with database or API).

Android best practices

Concurrency:

- Following operations **should NOT be done on Main/UI thread**.
 - Database access,
 - Reading/writing to files,
 - DataStore access,s
 - Server communication,
 - Calculations
 - AI inference,
 - etc.

Options: Coroutines (recommended), Handler, Threads, etc.

Android best practices

Configuration Changes

- Ensure the app handles configuration changes (e.g. device rotation, light/dark theme switch or language changes)
- Preserve the screen's state during configuration changes using `ViewModel`, `rememberSaveable` in Compose or `onSavedInstanceState` callback in Activity
- **Common issues to address:**
 - Prevent losing user input in fields during config changes
 - Prevent reloading data during config changes